

Django Backend Learning — Phase 1 Notes

Revision notes from our lessons • Django fundamentals + first DRF concepts

Purpose: This PDF is a revision guide, not a replacement for the lessons. It focuses on the concepts we have actually covered together, with simple explanations and practical examples.

1. The Big Picture

A backend receives requests from clients, processes them, often talks to a database, and sends a response back.

A simple mental model is:

```
Client → URL/Router → View → Logic/Serializer → Database → Response
```

Django gives us tools for the web application. Django REST Framework (DRF) adds tools for building APIs.

2. Django URLs

A URL pattern connects a URL to a view. Django reads the URL and decides which Python function or class should handle the request.

```
path('products/', views.products, name='products')
```

Dynamic values can be captured from the URL using converters:

```
path('products/', views.product_details, name='product_detail')
```

Here **int** is the converter and **product_id** is the parameter name. The view must use the same parameter name.

```
def product_details(request, product_id):
```

A common mistake is **<int:>**. It is invalid because there is no parameter name. The correct form is **<int:product_id>**.

Trailing slashes matter depending on the URL pattern and Django's APPEND_SLASH behavior. When testing APIs, use the exact route you defined to avoid confusing redirects or method-related behavior.

3. Django Views and HTTP Methods

A view receives a request and returns a response. The request contains information such as the HTTP method and path.

```
request.method  
request.path
```

Common HTTP methods we practiced:

GET — retrieve data

POST — create/send new data

PUT — full update/replacement

PATCH — partial update

DELETE — delete data

4. Returning JSON

For an API, we commonly return JSON rather than HTML.

Django's basic response class is `JsonResponse`:

```
return JsonResponse({"id": product_id, "name": "Laptop", "price": 800})
```

Later, with DRF, we used `Response` together with serializers. DRF is designed specifically for building APIs.

5. POST Data and CSRF

When you first sent a POST request, Django returned a 403 CSRF error. This happened because Django's CSRF protection expects a valid CSRF cookie/token for normal browser-based POST forms.

For learning API endpoints, we temporarily used `@csrf_exempt` in the simple Django view. This is useful for understanding request handling, but it should not be treated as a general production security solution.

In a real browser form, Django normally uses a `csrf_token` in the template. API authentication and CSRF behavior depend on how the API is being used.

6. Reading JSON Request Data

For a basic Django view receiving JSON, we used `request.body` and parsed it with Python's `json` module.

```
data = json.loads(request.body)
name = data['name']
price = data['price']
```

This works, but it also means we have to manually handle parsing, validation, missing fields, and errors. DRF serializers will later make this much cleaner.

7. Django Templates and Django Template Language

Django templates are mainly used to generate HTML pages. Django Template Language (DTL) lets a template display values and use simple template logic.

```
{{ name }}
{% if user %}
Welcome, {{ user }}
{% endif %}
```

We also configured a project-level templates folder. The important idea is that templates are for presentation/HTML, while a REST API normally returns structured data such as JSON.

8. Models and the Database

A Django model is a Python class that describes data stored in the database.

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.IntegerField()
```

We created models such as `Product`, `Customer`, `Order`, and `Profile` and used migrations to create/update the database schema.

makemigrations creates migration instructions from model changes. **migrate** applies those migrations to the database.

```
python manage.py makemigrations
python manage.py migrate
```

9. Model Relationships

We practiced relationships between models. For example, an Order can point to a Product and Customer using ForeignKey. A Profile can be connected one-to-one with a Customer.

ForeignKey is commonly used when many records can relate to one record. OneToOneField is used when one record corresponds to one related record.

The **related_name** option gives you a convenient reverse relationship name, such as `customer.orders.all()`.

10. Django ORM — First Concepts

The ORM lets Python code communicate with the database without writing SQL for every basic operation.

```
Product.objects.all()
Product.objects.get(id=1)
Product.objects.filter(price=100)
```

We will continue the ORM in more depth later because it is one of the most important connections between an API and a database.

11. DRF Serializers

A serializer helps convert Django model/queryset data into API-friendly data and helps validate incoming data before saving it.

Mental model:

```
Model object ↔ Serializer ↔ JSON/API data
```

With ModelSerializer, DRF can derive many serializer fields from a Django model.

12. Serializer Validation

We created field-level validation such as `validate_price()`. Validation is where we can reject invalid input before it reaches the database.

```
def validate_price(self, value):
    if value <= 0:
        raise serializers.ValidationError('Price must be greater than 0.')
    return value
```

A useful mental model is: request data → serializer validation → save only if valid.

13. APIView

APIView gives explicit control over HTTP methods.

```
class ProductView(APIView):
    def get(self, request):
        ...

    def post(self, request):
        ...
```

It is more structured than a simple function-based API view, while still letting you write the behavior yourself.

14. Generic Views

Generic views provide standard API behavior so you do not have to repeatedly write common CRUD code.

```
class ProductListView(ListCreateAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

ListCreateAPIView provides standard list and create behavior.

```
class ProductDetailView(RetrieveUpdateDestroyAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

RetrieveUpdateDestroyAPIView provides retrieve, update, and destroy behavior.

15. ViewSets

A ViewSet groups operations for one resource. Instead of thinking only in terms of get() and post(), ViewSets use actions such as list(), create(), retrieve(), partial_update(), and destroy().

```
GET /products/ → list()
POST /products/ → create()
GET /products/1/ → retrieve()
PATCH /products/1/ → partial_update()
DELETE /products/1/ → destroy()
```

16. Routers

A Router can generate URL patterns for a ViewSet.

```
router = DefaultRouter()
router.register('products', ProductViewSet, basename='product')
urlpatterns = router.urls
```

Mental model: **ViewSet defines API actions; Router maps URLs to those actions.**

17. ModelViewSet

ModelViewSet provides the standard CRUD actions for a model. That is why the final Product API became very small.

```
class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

The framework provides list, create, retrieve, update, partial_update, and destroy. You configure the queryset and serializer.

18. PUT vs PATCH

This distinction is important for API testing and design.

PUT — conceptually a full update/replacement. Required fields normally need to be supplied.

PATCH — partial update. Only the fields you want to change need to be supplied.

In DRF, PATCH uses partial=True internally, which allows fields not included in the request to remain unchanged.

```
PATCH /products/1/
{
  "price": 1000
}
```

For a normal serializer where name and price are required, a PUT containing only price can fail validation because name is missing.

19. The Backend Mental Model So Far

```
Client
↓
URL / Router
↓
View / ViewSet
↓
Serializer
↓
Validation
↓
ORM / Model
↓
Database
↓
Serializer
↓
HTTP Response
```

You should now recognize that each layer has a job: URLs route requests, views/ViewSets handle API operations, serializers convert and validate data, models describe database data, and the ORM communicates with the database.

20. Common Errors You Encountered

403 CSRF cookie not set — Django's CSRF protection rejected a POST request without the expected CSRF information.

URL route 'item/<int:>/' is invalid — the converter had no parameter name. Use a form such as `<int:item_id>`.

500 errors during API testing — several of your earlier issues were caused by URL/request mismatches. When debugging, check the terminal traceback and compare the exact URL with the URL pattern.

21. Quick Revision Checklist

- I understand request → URL → view → response.
- I know GET, POST, PUT, PATCH, and DELETE.
- I know how dynamic URL parameters work.
- I understand why trailing slashes can matter.

- I know what Django templates/DTL are used for.
- I understand models and migrations.
- I understand ForeignKey and OneToOneField at a basic level.
- I understand the purpose of the ORM.
- I understand serializers and validation.
- I understand APIView.
- I understand Generic Views.
- I understand ViewSets and Routers.
- I understand why ModelViewSet reduces CRUD boilerplate.
- I understand the conceptual difference between PUT and PATCH.

22. What Comes Next

The next major phase is **Authentication and Authorization**. Before that, we will continue strengthening the database/ORM connection where useful.